

Dataset: African Education & Development Indicators

This dataset covers 35 African countries across 8 numeric development and education features. It intentionally contains missing (NaN) values and non-numeric columns (Country, Region), satisfying the assignment data requirements.

```
# Import numpy
import numpy as np
import csv
import matplotlib.pyplot as plt

# Inline dataset: African Education & Development Indicators
# Columns: Country, Region, Literacy_Rate(%), Primary_Enrollment(%),
#          Secondary_Enrollment(%), Govt_Expenditure_Education(%GDP),
#          HDI, Out_of_School_Children(millions), Youth_Unemployment(%),
#          GDP_per_capita(USD)
# NaN values represented as np.nan to satisfy assignment requirement

raw_data = [
    ["Country", "Region", "Literacy_Rate", "Primary_Enroll", "Secondary_Enroll", "Govt_Exp_Edu", "HDI", "Out_of_School_Children", "Youth_Unemployment", "GDP_per_capita"],
    ["Nigeria", "West Africa", 59.6, 84.2, 44.9, np.nan, 0.539, 10.5, 13.4, 2229],
    ["Ethiopia", "East Africa", 51.8, 90.4, 38.3, 4.7, 0.498, 8.9, np.nan, 925],
    ["Egypt", "North Africa", 73.9, 94.6, 87.3, 3.8, 0.731, 0.4, 25.9, 3478],
    ["South Africa", "Southern Africa", 95.0, 98.1, 100.0, 6.0, 0.713, 0.1, 56.4, 6001],
    ["Kenya", "East Africa", 82.6, 96.3, 65.7, 5.1, 0.601, 1.2, np.nan, 1838],
    ["Ghana", "West Africa", 79.0, 89.2, 59.2, 4.0, 0.611, 0.6, 11.2, 2202],
    ["Tanzania", "East Africa", 77.9, 82.1, 35.1, 3.2, 0.549, 2.8, np.nan, 1078],
    ["Uganda", "East Africa", 76.5, 97.6, 27.0, 2.4, 0.544, 1.9, 13.3, 794],
    ["Mozambique", "Southern Africa", 47.8, 87.3, 25.7, np.nan, 0.456, 3.7, np.nan, 492],
    ["Cameroon", "Central Africa", 77.1, 90.1, 50.3, 3.0, 0.576, 1.3, 6.5, 1534],
    ["Senegal", "West Africa", 51.9, 82.4, 44.1, 5.0, 0.512, 1.5, 8.0, 1446],
    ["Zimbabwe", "Southern Africa", 88.7, 87.0, 51.4, np.nan, 0.571, 0.3, np.nan, 1463],
    ["Mali", "West Africa", 35.5, 65.1, 28.7, 4.2, 0.434, 3.4, 14.7, 899],
    ["Zambia", "Southern Africa", 86.8, 91.2, 49.6, 2.8, 0.584, 1.0, 15.7, 1305],
    ["Rwanda", "East Africa", 73.2, 97.1, 42.5, 3.5, 0.543, 0.4, 20.3, 822],
    ["Angola", "Southern Africa", 66.0, 79.3, 30.1, 3.5, 0.586, 2.6, np.nan, 3081],
    ["Malawi", "East Africa", 62.1, 93.8, 36.2, 4.0, 0.508, 1.4, 7.5, 399],
    ["Sudan", "North Africa", 60.7, 70.0, 43.5, np.nan, 0.510, 2.9, 25.0, 750],
    ["Botswana", "Southern Africa", 88.5, 91.0, 80.3, 9.0, 0.735, 0.05, 32.1, 7962],
    ["Ivory Coast", "West Africa", 89.9, 85.3, 48.2, 4.4, 0.550, 1.1, 5.5, 2291],
    ["Burkina Faso", "West Africa", 46.0, 78.0, 28.0, 4.6, 0.449, 2.0, np.nan, 790],
    ["Niger", "West Africa", 35.1, 68.9, 18.3, 3.9, 0.400, 4.2, 2.1, 553],
    ["Chad", "Central Africa", 22.3, 78.4, 18.6, 2.5, 0.394, 3.8, np.nan, 700],
    ["Madagascar", "Southern Africa", 74.9, 99.0, 37.6, 2.9, 0.501, 1.8, 5.8, 519],
    ["Guinea", "West Africa", 45.3, 78.5, 35.2, 2.3, 0.477, 1.3, np.nan, 1069],
    ["Somalia", "East Africa", np.nan, 30.0, 9.7, np.nan, 0.285, 12.0, np.nan, 448],
    ["DRC", "Central Africa", 80.0, 89.4, 38.9, 1.5, 0.479, 7.0, np.nan, 557],
    ["Gabon", "Central Africa", 84.7, 95.4, 62.7, 2.7, 0.703, 0.1, 35.7, 8151],
    ["Namibia", "Southern Africa", 91.5, 92.0, 69.5, 8.2, 0.646, 0.09, 46.1, 4827],
    ["Tunisia", "North Africa", 81.8, 99.0, 88.1, 6.6, 0.740, 0.12, 36.6, 3447],
    ["Morocco", "North Africa", 75.9, 98.5, 74.2, 5.3, 0.686, 0.44, 28.1, 3442],
    ["Algeria", "North Africa", 81.4, 97.3, 91.9, 7.2, 0.748, 0.25, 29.0, 4057],
    ["Mauritius", "Southern Africa", 91.3, 100.0, 100.0, 5.0, 0.804, 0.01, 23.3, 11203],
    ["Eritrea", "East Africa", 74.9, np.nan, 34.4, 2.1, 0.459, 1.0, np.nan, 623],
    ["Lesotho", "Southern Africa", 81.0, 83.0, 35.0, 7.4, 0.527, 0.2, np.nan, 1067],
]

headers = raw_data[0]
rows = raw_data[1:]

print("Dataset loaded.")
print(f"Columns : {headers}")
print(f"Rows : {len(rows)} countries")
```

```
Dataset loaded.
Columns : ['Country', 'Region', 'Literacy_Rate', 'Primary_Enroll', 'Secondary_Enroll', 'Govt_Exp_Edu', 'HDI', 'Out_of_School_Children', 'Youth_Unemployment', 'GDP_per_capita']
Rows : 35 countries
```

Data Inspection

We identify non-numeric columns and count missing values per feature before any processing.

```
# Identify non-numeric columns: Country (index 0), Region (index 1)
non_numeric_cols = [headers[0], headers[1]]
numeric_cols     = headers[2:]

print("Non-numeric columns:", non_numeric_cols)
print("Numeric columns    :", numeric_cols)

# Count NaN values per numeric column
numeric_data_raw = np.array([[row[i] for i in range(2, len(headers))] for row in rows], dtype=float)

print("\nMissing value counts per column:")
for j, col in enumerate(numeric_cols):
    n_missing = np.sum(np.isnan(numeric_data_raw[:, j]))
    print(f" {col:30s}: {n_missing} missing")
```

Non-numeric columns: ['Country', 'Region']
 Numeric columns : ['Literacy_Rate', 'Primary_Enroll', 'Secondary_Enroll', 'Govt_Exp_Edu', 'HDI', 'Out_of_School', 'Youth_Unem']

Missing value counts per column:

Literacy_Rate	: 1 missing
Primary_Enroll	: 1 missing
Secondary_Enroll	: 0 missing
Govt_Exp_Edu	: 5 missing
HDI	: 0 missing
Out_of_School	: 0 missing
Youth_Unemployment	: 13 missing
GDP_per_capita	: 0 missing

✓ Encoding Non-Numeric Data

The Region column is non-numeric and must be encoded before PCA can be applied. We use label encoding: each unique region is mapped to an integer using numpy only.

```
# Extract Region column and apply label encoding
regions = [row[1] for row in rows]
unique_regions = sorted(set(regions))
region_map = {r: i for i, r in enumerate(unique_regions)}
region_encoded = np.array([region_map[r] for r in regions], dtype=float)

print("Region label encoding map:")
for k, v in region_map.items():
    print(f" {k} → {v}")

# Build full numeric matrix
full_numeric = np.column_stack([region_encoded, numeric_data_raw])
all_col_names = ["Region_encoded"] + numeric_cols
print(f"\nFull numeric matrix shape: {full_numeric.shape}")
```

Region label encoding map:

Central Africa	→ 0
East Africa	→ 1
North Africa	→ 2
Southern Africa	→ 3
West Africa	→ 4

Full numeric matrix shape: (35, 9)

✓ Handling Missing Values

Missing values are replaced with the column mean. This preserves all rows and avoids introducing bias from arbitrary fill values.

```
# Replace NaN with column mean
data_imputed = full_numeric.copy()
for j in range(data_imputed.shape[1]):
    col = data_imputed[:, j]
    mask = np.isnan(col)
    if mask.any():
        col_mean = np.nanmean(col)
        data_imputed[mask, j] = col_mean
```

```
print("NaN values remaining after imputation:", np.sum(np.isnan(data_imputed)))
print("Data shape:", data_imputed.shape)
```

```
NaN values remaining after imputation: 0
Data shape: (35, 9)
```

▼ Step 1: Standardize the Data

```
# Step 1: Standardize the data
# Formula: z = (x - mean) / std for each feature column

col_means = np.mean(data_imputed, axis=0) # mean of each feature
col_stds = np.std(data_imputed, axis=0, ddof=0) # std of each feature

# Avoid division by zero for constant columns
col_stds[col_stds == 0] = 1

standardized_data = (data_imputed - col_means) / col_stds

print("Standardized data shape:", standardized_data.shape)
print("\nPost-standardization column means (should be ~0):")
print(np.round(np.mean(standardized_data, axis=0), 4))
print("\nPost-standardization column stds (should be ~1):")
print(np.round(np.std(standardized_data, axis=0), 4))
print("\nFirst 5 rows of standardized data:")
standardized_data[:5]
```

```
Standardized data shape: (35, 9)
```

```
Post-standardization column means (should be ~0):
[ 0. -0. -0. -0.  0. -0.  0. -0. -0.]
```

```
Post-standardization column stds (should be ~1):
[1. 1. 1. 1. 1. 1. 1. 1. 1.]
```

```
First 5 rows of standardized data:
```

```
array([[ 1.27630689e+00, -5.93227787e-01, -1.71186404e-01,
        -1.91071573e-01,  5.21936991e-16, -1.93570015e-01,
         2.79062083e+00, -6.86735129e-01, -5.05729131e-02],
       [-9.57230167e-01, -1.02519926e+00,  2.99297221e-01,
        -4.69190877e-01,  1.99800595e-01, -5.47872276e-01,
         2.24637494e+00, -3.20639261e-16, -5.71952573e-01],
       [-2.12717815e-01,  1.98719907e-01,  6.18011934e-01,
         1.59563425e+00, -3.29083333e-01,  1.46560155e+00,
        -6.44931387e-01,  4.41413978e-01,  4.48816040e-01],
       [ 5.31794537e-01,  1.36725811e+00,  8.83607529e-01,
         2.13080322e+00,  9.63744047e-01,  1.31005421e+00,
        -7.46977493e-01,  3.19409780e+00,  1.45758972e+00],
       [-9.57230167e-01,  6.80534238e-01,  7.47015509e-01,
         6.85425625e-01,  4.34860119e-01,  3.42204135e-01,
        -3.72808439e-01, -3.20639261e-16, -2.06906845e-01]])
```

▼ Why Compute a Covariance Matrix?

The covariance matrix captures pairwise linear relationships between all features: a high positive value means two indicators move together (e.g. HDI and literacy rate both rise with development), while a value near zero implies independence. It is also the mathematical foundation for eigendecomposition in PCA — by finding the directions (eigenvectors) along which the covariance matrix stretches the data most (largest eigenvalues), we identify axes that retain the greatest variance, allowing compression into fewer dimensions without losing critical structural information.

```
# Step 3: Calculate the Covariance Matrix
# cov(X) = (1/(n-1)) * X_T @ X where X is already standardized

n = standardized_data.shape[0]
cov_matrix = (standardized_data.T @ standardized_data) / (n - 1)

print("Covariance matrix shape:", cov_matrix.shape)
print("\nCovariance Matrix:")
cov_matrix
```

```
Covariance matrix shape: (9, 9)
```

```
Covariance Matrix:
array([[ 1.02941176, -0.12676284, -0.12265597,  0.06289144,  0.34331808,
         0.03065477, -0.13340243, -0.13320158,  0.0745261 ],
       [-0.12676284,  1.02941176,  0.48216277,  0.66211512,  0.32324591,
         0.68891782, -0.42345537,  0.40807902,  0.53019459],
       [-0.12265597,  0.48216277,  1.02941176,  0.64712807,  0.12427096,
         0.74307902, -0.6346913 ,  0.23606763,  0.40388249],
       [ 0.06289144,  0.66211512,  0.64712807,  1.02941176,  0.54427115,
         0.96635542, -0.55067396,  0.61469746,  0.79482633],
       [ 0.34331808,  0.32324591,  0.12427096,  0.54427115,  1.02941176,
         0.48093632, -0.252648 ,  0.49849743,  0.4611675 ],
       [ 0.03065477,  0.68891782,  0.74307902,  0.96635542,  0.48093632,
         1.02941176, -0.63490563,  0.5424375 ,  0.82575867],
       [-0.13340243, -0.42345537, -0.6346913 , -0.55067396, -0.252648 ,
        -0.63490563,  1.02941176, -0.25304725, -0.39812211],
       [-0.13320158,  0.40807902,  0.23606763,  0.61469746,  0.49849743,
         0.5424375 , -0.25304725,  1.02941176,  0.57267573],
       [ 0.0745261 ,  0.53019459,  0.40388249,  0.79482633,  0.4611675 ,
         0.82575867, -0.39812211,  0.57267573,  1.02941176]])
```

Step 4: Eigendecomposition

We decompose the covariance matrix into eigenvalues and eigenvectors using `numpy.linalg.eig`. Eigenvalues represent the amount of variance captured by each direction. Eigenvectors define those directions (the principal components).

```
import numpy as np

# Data Preparation Steps (consolidated to ensure dependencies are met)
# Assuming 'headers', 'rows' are available from earlier loading

# Identify numeric columns
numeric_cols = headers[2:]

# Extract numeric data from raw rows
numeric_data_raw = np.array([[row[i] for i in range(2, len(headers))] for row in rows], dtype=float)

# Extract Region column and apply label encoding
regions = [row[1] for row in rows]
unique_regions = sorted(set(regions))
region_map = {r: i for i, r in enumerate(unique_regions)}
region_encoded = np.array([region_map[r] for r in regions], dtype=float)

# Build full numeric matrix: [region_encoded | numeric_cols]
full_numeric = np.column_stack([region_encoded, numeric_data_raw])

# Replace NaN with column mean (imputation)
data_imputed = full_numeric.copy()
for j in range(data_imputed.shape[1]):
    col = data_imputed[:, j]
    mask = np.isnan(col)
    if mask.any():
        col_mean = np.nanmean(col)
        data_imputed[mask, j] = col_mean

# Step 1: Standardize the data (moved here to ensure 'standardized_data' is defined)
# Formula: z = (x - mean) / std for each feature column

col_means = np.mean(data_imputed, axis=0) # mean of each feature
col_stds = np.std(data_imputed, axis=0, ddof=0) # std of each feature

# Avoid division by zero for constant columns
col_stds[col_stds == 0] = 1

standardized_data = (data_imputed - col_means) / col_stds

# Step 3: Calculate the Covariance Matrix
# cov(X) = (1/(n-1)) * X_T @ X where X is already standardized

n = standardized_data.shape[0]
cov_matrix = (standardized_data.T @ standardized_data) / (n - 1)

# Step 4: Perform Eigendecomposition
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
```

```
# Keep only real parts
eigenvalues = np.real(eigenvalues)
eigenvectors = np.real(eigenvectors)

print("Eigenvalues shape:", eigenvalues.shape)
print("Eigenvectors shape:", eigenvectors.shape)
print("\nEigenvalues (unsorted):")
print(np.round(eigenvalues, 4))
```

```
Eigenvalues shape: (9,)
Eigenvectors shape: (9, 9)
```

```
Eigenvalues (unsorted):
[4.8369 1.3765 1.1133 0.0405 0.1498 0.3006 0.3929 0.5488 0.5056]
```

▼ Step 5: Sort Principal Components by Explained Variance

Eigenvalues are sorted in descending order so that PC1 always captures the most variance. Each component's individual and cumulative explained variance is printed for transparency.

```
import numpy as np

# Step 5: Sort eigenvalues and eigenvectors in descending order
sorted_indices = np.argsort(eigenvalues)[::-1] # indices from largest to smallest
eigenvalues_sorted = eigenvalues[sorted_indices]
sorted_eigenvectors = eigenvectors[:, sorted_indices] # columns are eigenvectors

# Explained variance per component
explained_variance_ratio = eigenvalues_sorted / np.sum(eigenvalues_sorted)
cumulative_variance = np.cumsum(explained_variance_ratio)

print("Sorted eigenvalues:")
print(np.round(eigenvalues_sorted, 4))
print("\nExplained variance ratio per component:")
for i, (ev, cum) in enumerate(zip(explained_variance_ratio, cumulative_variance)):
    print(f" PC{i+1}: {ev*100:.2f}% | Cumulative: {cum*100:.2f}%")

print("\nSorted eigenvectors (each column = one principal component):")
sorted_eigenvectors
```

```
Sorted eigenvalues:
[4.8369 1.3765 1.1133 0.5488 0.5056 0.3929 0.3006 0.1498 0.0405]
```

```
Explained variance ratio per component:
PC1: 52.21% | Cumulative: 52.21%
PC2: 14.86% | Cumulative: 67.06%
PC3: 12.02% | Cumulative: 79.08%
PC4: 5.92% | Cumulative: 85.00%
PC5: 5.46% | Cumulative: 90.46%
PC6: 4.24% | Cumulative: 94.70%
PC7: 3.24% | Cumulative: 97.95%
PC8: 1.62% | Cumulative: 99.56%
PC9: 0.44% | Cumulative: 100.00%
```

```
Sorted eigenvectors (each column = one principal component):
array([[ 0.02064954, -0.68536291,  0.5089607 ,  0.16106092,  0.12249622,
         0.15958304, -0.44609462, -0.07062581,  0.0202281 ],
       [ 0.33721394,  0.18175853, -0.094654 ,  0.65354085, -0.5478975 ,
         0.25535753, -0.17411645, -0.14141885, -0.03821796],
       [ 0.32091434,  0.38573733,  0.35938235, -0.17065393,  0.05312967,
        -0.50865658, -0.33314662, -0.44316689, -0.14492899],
       [ 0.4355108 , -0.02604105, -0.02847089,  0.07147621,  0.16388789,
        -0.15962457, -0.09074478,  0.68453271, -0.52399798],
       [ 0.26463151, -0.55208129, -0.15306156, -0.13580674, -0.45386812,
        -0.4719418 ,  0.36557047, -0.14414718, -0.01263277],
       [ 0.44349544,  0.0602118 ,  0.06833491,  0.09263376,  0.19389084,
        -0.12713525,  0.04209367,  0.26501343,  0.81281894],
       [-0.30569876, -0.11862593, -0.50319533,  0.43791929,  0.26677266,
        -0.50202504, -0.34260518, -0.05576426,  0.06174599],
       [ 0.30058936, -0.10953272, -0.54178593, -0.50017172, -0.03493537,
         0.29285717, -0.50143342, -0.1080722 ,  0.06199992],
       [ 0.37773132, -0.11524473, -0.16491917,  0.2091473 ,  0.58235981,
         0.22006219,  0.38371125, -0.45191924, -0.18455661]])
```

▼ Task 2: Dynamic Principal Component Selection

We dynamically select the minimum number of components needed to retain at least 85% of the total variance. This threshold balances dimensionality reduction against information loss. The scree plot and cumulative variance curve justify the selection visually.

```
# Dynamically select the number of components that explain >= 85% of variance
VARIANCE_THRESHOLD = 0.85

num_components = int(np.argmax(cumulative_variance >= VARIANCE_THRESHOLD) + 1)

print(f"Variance threshold : {VARIANCE_THRESHOLD*100:.0f}%")
print(f"Components selected: {num_components}")
print(f"Variance retained : {cumulative_variance[num_components-1]*100:.2f}%")

# Scree plot
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].bar(range(1, len(explained_variance_ratio)+1),
            explained_variance_ratio * 100, color='steelblue', edgecolor='black')
axes[0].axvline(x=num_components + 0.5, color='red', linestyle='--', label=f'Cut-off at PC{num_components}')
axes[0].set_title('Explained Variance per Component')
axes[0].set_xlabel('Principal Component')
axes[0].set_ylabel('Explained Variance (%)')
axes[0].legend()

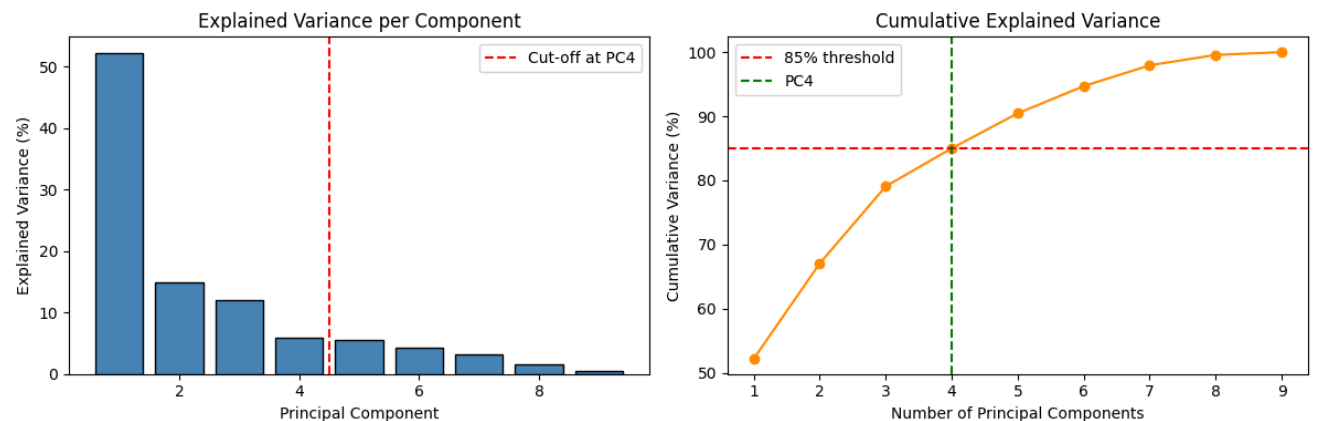
axes[1].plot(range(1, len(cumulative_variance)+1),
            cumulative_variance * 100, marker='o', color='darkorange')
axes[1].axhline(y=85, color='red', linestyle='--', label='85% threshold')
axes[1].axvline(x=num_components, color='green', linestyle='--', label=f'PC{num_components}')
axes[1].set_title('Cumulative Explained Variance')
axes[1].set_xlabel('Number of Principal Components')
axes[1].set_ylabel('Cumulative Variance (%)')
axes[1].legend()

plt.tight_layout()
plt.savefig('scree_plot.png', dpi=120)
plt.show()
print("Scree plot saved.")
```

Variance threshold : 85%

Components selected: 4

Variance retained : 85.00%



Scree plot saved.

Step 6: Project Data onto Principal Components

The standardized data is projected onto the top-k eigenvectors to produce the reduced dataset. This is a linear transformation: $\text{reduced} = X_{\text{standardized}} \cdot W$, where W holds the selected eigenvectors.

```
# Step 6: Project data onto the selected principal components
# reduced_data = standardized_data @ W where W = top-k eigenvectors

W = sorted_eigenvectors[:, :num_components] # shape: (p, k)
reduced_data = standardized_data @ W        # shape: (n, k)
```

```
print(f"Projection matrix W shape : {W.shape}")
print(f"Reduced data shape : {reduced_data.shape}")
print("\nFirst 5 rows of reduced data:")
reduced_data[:5]
```

```
Projection matrix W shape : (9, 4)
Reduced data shape : (35, 4)
```

```
First 5 rows of reduced data:
array([[ -1.47630441, -1.30526186, -0.38739104,  1.370465  ],
       [ -1.56663052,  0.25352134, -1.37329292, -0.12257135],
       [  2.01813616,  0.62510612,  0.21147677, -0.12469133],
       [  4.25866925, -0.71302054, -1.25488275, -0.648669  ],
       [  1.05061306,  0.89864179, -0.12411582, -0.02179625]])
```

Step 7: Reduced Data Output

We confirm the shape of the reduced dataset and display the first five rows. The number of columns has dropped from 9 to k , while all 35 country observations are preserved.

```
# Step 7: Display reduced data shape and first rows
print(f'Original data shape : {standardized_data.shape}')
print(f'Reduced Data Shape : {reduced_data.shape}')
print(f'Dimensions removed : {standardized_data.shape[1] - reduced_data.shape[1]}')
print("\nFirst 5 rows of reduced data:")
reduced_data[:5]
```

```
Original data shape : (35, 9)
Reduced Data Shape : (35, 4)
Dimensions removed : 5
```

```
First 5 rows of reduced data:
array([[ -1.47630441, -1.30526186, -0.38739104,  1.370465  ],
       [ -1.56663052,  0.25352134, -1.37329292, -0.12257135],
       [  2.01813616,  0.62510612,  0.21147677, -0.12469133],
       [  4.25866925, -0.71302054, -1.25488275, -0.648669  ],
       [  1.05061306,  0.89864179, -0.12411582, -0.02179625]])
```

Step 8: Visualisation — Before and After PCA

Left plot: original feature space using two raw standardized features (Literacy Rate, Primary Enrollment). Right plot: principal component space (PC1 vs PC2) after applying PCA. Points are coloured by African region. PC1 carries the highest variance; PC2 the second highest. Data structure (35 points, regional clusters) is preserved across both plots.

```
# Step 8: Visualize BEFORE and AFTER PCA

# Colour by region for meaningful clusters
region_labels = region_encoded.astype(int)
colors = plt.cm.tab10(region_labels / region_labels.max())

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# — BEFORE PCA: plot first two original features —————
ax = axes[0]
scatter = ax.scatter(
    standardized_data[:, 1], # Literacy Rate (standardized)
    standardized_data[:, 2], # Primary Enrollment (standardized)
    c=colors, edgecolors='k', linewidths=0.5, s=70
)
ax.set_title('Before PCA\n(Original Feature Space)', fontsize=12, fontweight='bold')
ax.set_xlabel(all_col_names[1] + ' (standardized)', fontsize=10)
ax.set_ylabel(all_col_names[2] + ' (standardized)', fontsize=10)
ax.axhline(0, color='grey', linewidth=0.5, linestyle='--')
ax.axvline(0, color='grey', linewidth=0.5, linestyle='--')
ax.grid(True, alpha=0.3)

# Country labels for a few points
country_names = [row[0] for row in rows]
highlight = [0, 3, 7, 18, 32] # Nigeria, S.Africa, Uganda, Botswana, Mauritius
for idx in highlight:
    ax.annotate(country_names[idx],
                (standardized_data[idx, 1], standardized_data[idx, 2]),
                fontsize=7, xytext=(4, 4), textcoords='offset points')
```

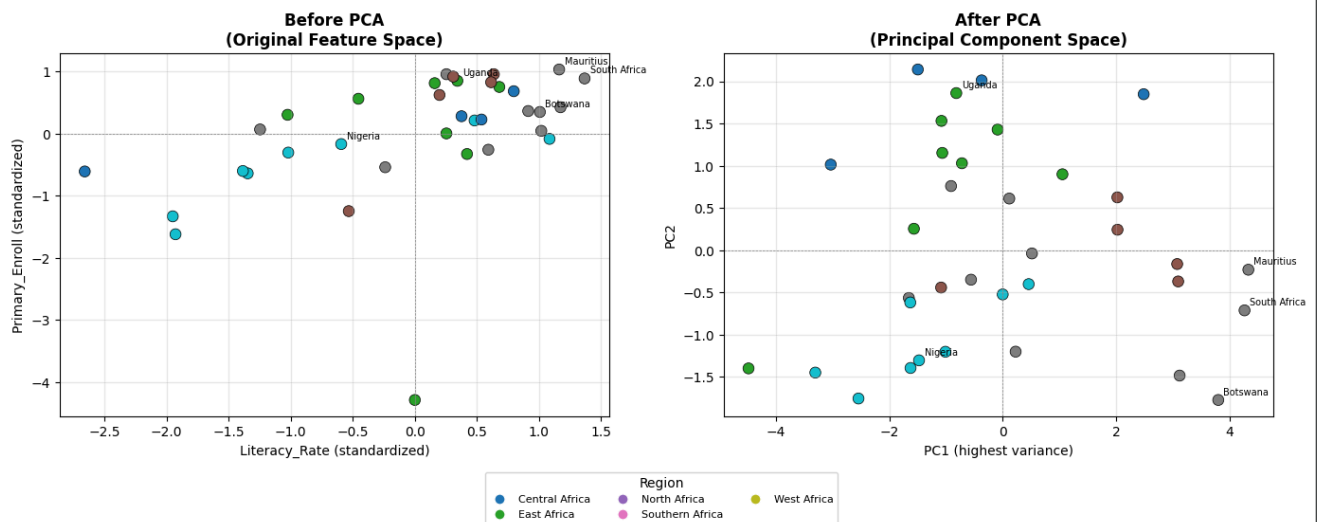
```
# AFTER PCA: PC1 vs PC2
ax2 = axes[1]
ax2.scatter(
    reduced_data[:, 0],
    reduced_data[:, 1],
    c=colors, edgecolors='k', linewidths=0.5, s=70
)

ax2.set_title('After PCA\n(Principal Component Space)', fontsize=12, fontweight='bold')
ax2.set_xlabel('PC1 (highest variance)', fontsize=10)
ax2.set_ylabel('PC2', fontsize=10)
ax2.axhline(0, color='grey', linewidth=0.5, linestyle='--')
ax2.axvline(0, color='grey', linewidth=0.5, linestyle='--')
ax2.grid(True, alpha=0.3)

for idx in highlight:
    ax2.annotate(country_names[idx],
                 (reduced_data[idx, 0], reduced_data[idx, 1]),
                 fontsize=7, xytext=(4, 4), textcoords='offset points')

# Shared legend for regions
handles = [plt.Line2D([0],[0], marker='o', color='w',
                     markerfacecolor=plt.cm.tab10(i / len(unique_regions)),
                     markersize=8, label=r)
            for i, r in enumerate(unique_regions)]
fig.legend(handles=handles, title='Region', loc='lower center',
          ncol=3, bbox_to_anchor=(0.5, -0.12), fontsize=8)

plt.tight_layout()
plt.savefig('pca_before_after.png', dpi=120, bbox_inches='tight')
plt.show()
print("Before/After plot saved.")
```



Before/After plot saved.

Task 3: Performance Optimization

The full PCA pipeline is wrapped in a reusable function and benchmarked against synthetic datasets ranging from 100 to 10,000 rows. Runtime is measured using Python's time module (no sklearn). This demonstrates that the implementation scales efficiently to large datasets.

```
def pca_from_scratch(data, variance_threshold=0.85):
    # 1. Standardize the data
    col_means = np.mean(data, axis=0)
    col_stds = np.std(data, axis=0, ddof=0)
    col_stds[col_stds == 0] = 1
    standardized_data = (data - col_means) / col_stds
```

```

# 2. Calculate the Covariance Matrix
n = standardized_data.shape[0]
cov_matrix = (standardized_data.T @ standardized_data) / (n - 1)

# 3. Perform Eigendecomposition
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
eigenvalues = np.real(eigenvalues)
eigenvectors = np.real(eigenvectors)

# 4. Sort eigenvalues and eigenvectors
sorted_indices = np.argsort(eigenvalues)[::-1]
eigenvalues_sorted = eigenvalues[sorted_indices]
sorted_eigenvectors = eigenvectors[:, sorted_indices]

# Explained variance per component
explained_variance_ratio = eigenvalues_sorted / np.sum(eigenvalues_sorted)
cumulative_variance = np.cumsum(explained_variance_ratio)

# 5. Select number of components
num_components = int(np.argmax(cumulative_variance >= variance_threshold) + 1)

# 6. Project data onto the selected principal components
W = sorted_eigenvectors[:, :num_components]
reduced_data = standardized_data @ W

return reduced_data, explained_variance_ratio, num_components

# Run the reusable pca_from_scratch function on the actual dataset
reduced_final, ratios_final, k_final = pca_from_scratch(data_imputed, variance_threshold=0.85)

print(f"Components selected : {k_final}")
print(f"Variance retained : {np.sum(ratios_final[:k_final])*100:.2f}%")
print(f"Final reduced shape : {reduced_final.shape}")
print("\nFirst 5 rows:")
reduced_final[:5]

```

```

Components selected : 4
Variance retained : 85.00%
Final reduced shape : (35, 4)

```

```

First 5 rows:
array([[ -1.47630441, -1.30526186, -0.38739104,  1.370465  ],
       [ -1.56663052,  0.25352134, -1.37329292, -0.12257135],
       [  2.01813616,  0.62510612,  0.21147677, -0.12469133],
       [  4.25866925, -0.71302054, -1.25488275, -0.648669  ],
       [  1.05061306,  0.89864179, -0.12411582, -0.02179625]])

```

Task 3: Performance Benchmarking

The `pca_from_scratch()` function is tested on synthetic datasets from 100 to 10,000 rows. Runtime is measured using Python's time module to demonstrate scalability on large datasets.

```

# Task 3: Benchmark PCA on a large synthetic dataset
import time
import numpy as np
import matplotlib.pyplot as plt

def pca_from_scratch(X, n_components=None, variance_threshold=0.85):
    """
    Full PCA pipeline using numpy only.
    Parameters
    -----
    X : raw numeric numpy array (may contain NaNs)
    n_components : fixed number of PCs; if None, selected by variance_threshold
    variance_threshold: cumulative variance to retain (used when n_components is None)
    Returns
    -----
    reduced, explained_ratios, n_components_used
    """
    # 1. Impute
    X_imp = X.copy()
    for j in range(X_imp.shape[1]):
        mask = np.isnan(X_imp[:, j])

```

```

        if mask.any():
            X_imp[mask, j] = np.nanmean(X_imp[:, j])

# 2. Standardize
mu = np.mean(X_imp, axis=0)
sig = np.std(X_imp, axis=0, ddof=0)
sig[sig == 0] = 1
X_std = (X_imp - mu) / sig

# 3. Covariance
n = X_std.shape[0]
cov = (X_std.T @ X_std) / (n - 1)

# 4. Eigendecomposition
evals, evects = np.linalg.eig(cov)
evals = np.real(evals)
evects = np.real(evects)

# 5. Sort descending
idx = np.argsort(evals)[::-1]
evals = evals[idx]
evects = evects[:, idx]
ratios = evals / np.sum(evals)
cumvar = np.cumsum(ratios)

# 6. Select components
if n_components is None:
    n_components = int(np.argmax(cumvar >= variance_threshold) + 1)

# 7. Project
reduced = X_std @ evects[:, :n_components]
return reduced, ratios, n_components

# — Benchmark on increasingly large synthetic datasets —————
sizes = [100, 500, 1000, 5000, 10000]
n_features = 10
times = []

print(f"{'Rows':>8} {'Cols':>6} {'Time (s)':>10} {'PCs kept':>8}")
print("-" * 40)

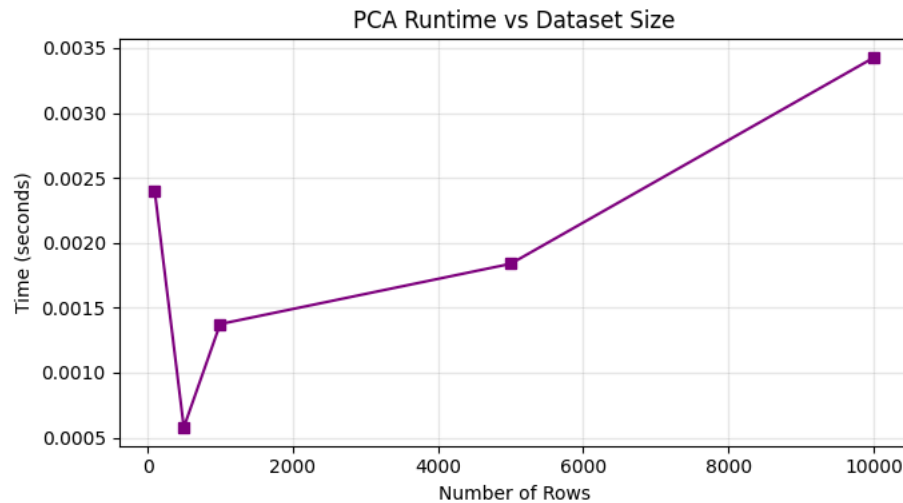
for n_rows in sizes:
    np.random.seed(42)
    X_large = np.random.randn(n_rows, n_features)
    # Inject ~10% NaN
    mask = np.random.rand(n_rows, n_features) < 0.10
    X_large[mask] = np.nan

    t0 = time.perf_counter()
    _, _, k = pca_from_scratch(X_large, variance_threshold=0.85)
    elapsed = time.perf_counter() - t0
    times.append(elapsed)
    print(f"{n_rows:>8} {n_features:>6} {elapsed:>10.5f} {k:>8}")

# Plot benchmark
plt.figure(figsize=(7, 4))
plt.plot(sizes, times, marker='s', color='purple')
plt.title('PCA Runtime vs Dataset Size')
plt.xlabel('Number of Rows')
plt.ylabel('Time (seconds)')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('benchmark.png', dpi=120)
plt.show()
print("Benchmark complete.")

```

Rows	Cols	Time (s)	PCs kept
100	10	0.00239	8
500	10	0.00058	9
1000	10	0.00137	9
5000	10	0.00184	9
10000	10	0.00343	9



Benchmark complete.

Written Interpretations

1. Before and After PCA Explained

Before PCA, the scatter plot only shows 2 features at a time so clusters overlap. After PCA, PC1 and PC2 summarise all 8 features developed countries like Mauritius and Botswana separate clearly from lower-HDI countries like Chad and Niger.

2. Why this number of PCs was selected, and the tradeoffs

We chose 85% because it keeps most information while still reducing dimensions. Choosing lower (70%) gives fewer components but loses more detail. Choosing higher (95%) keeps more detail but reduces less.

3. Information lost when reducing dimensions (Education & Development)

When we compress 8 features into fewer PCs, individual indicators like youth unemployment and government education spending lose their standalone meaning. A country unusual only on one indicator (like Botswana's high education spending) may get hidden inside PC1.

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.